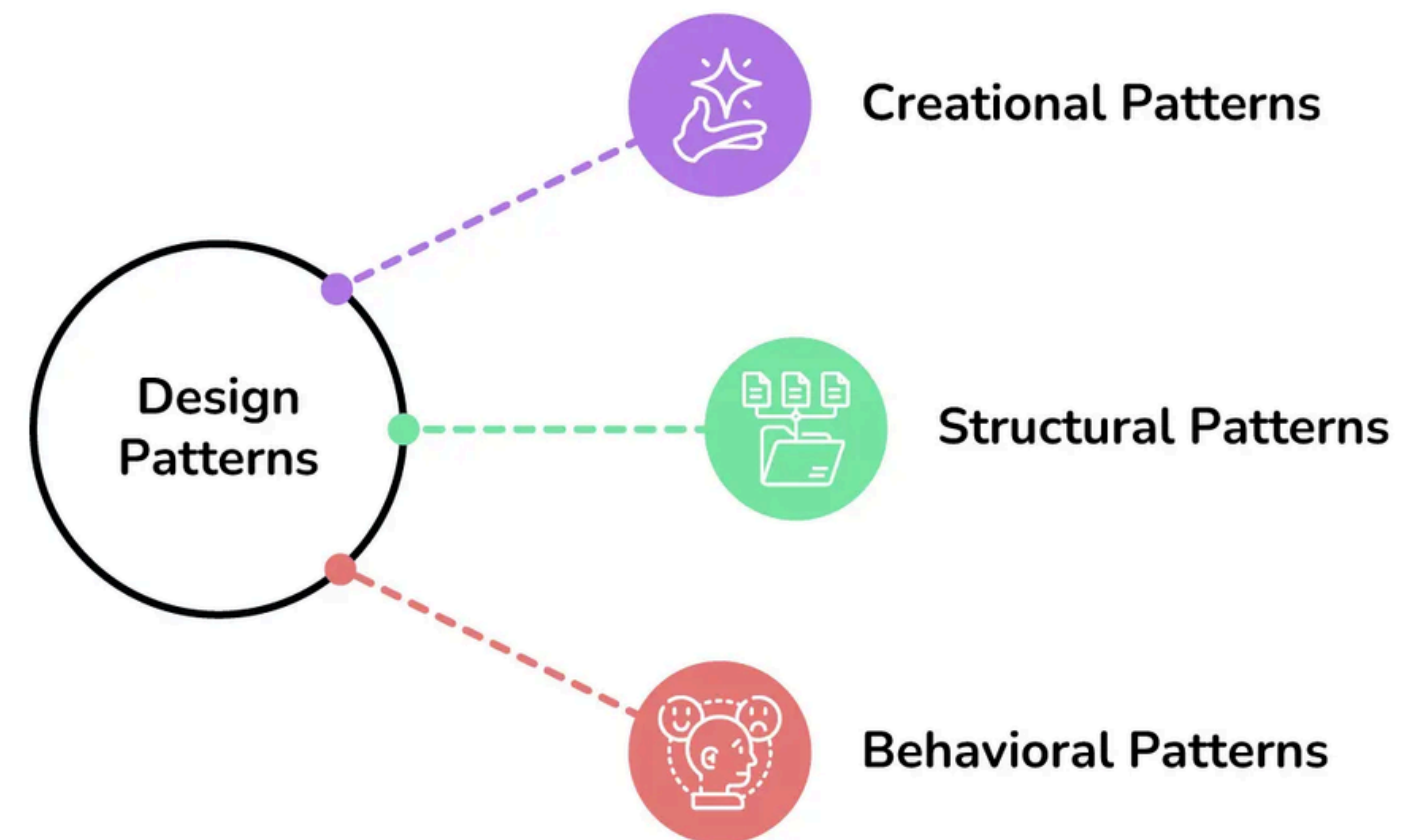


Team 3

DESIGN PATTERNS

MEMBERS:

Nguyễn Quốc Hoàng Pháp
Nguyễn Minh Nhân
Nguyễn Võ Cường Thịnh
Nguyễn Đại Nam
Mai Tuấn Vinh



SE1928

WHAT IS DESIGN PATTERN

Definition

A reusable, standardized blueprint for solving commonly recurring problems in code design

Nature

Not a specific piece of code, but a blueprint or template on how to solve a problem

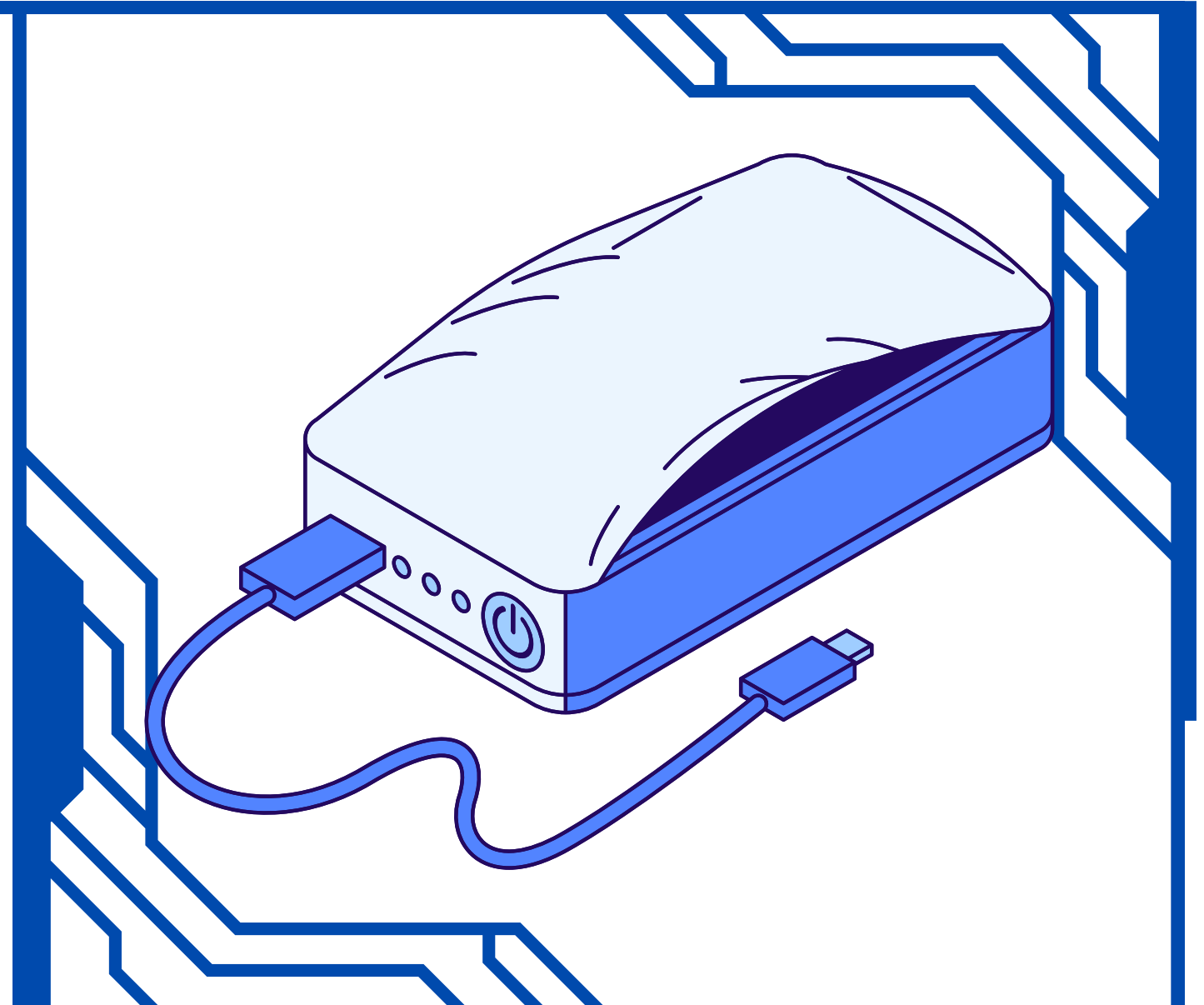


CREATIONAL DESIGN PATTERNS

Deals with object creation

Main Purpose:

- How to create objects flexibly
- Hide instantiation logic
- Ensure scalability
- Avoid hard-coding.



5 PATTERNS

1. BUILDER PATTERN

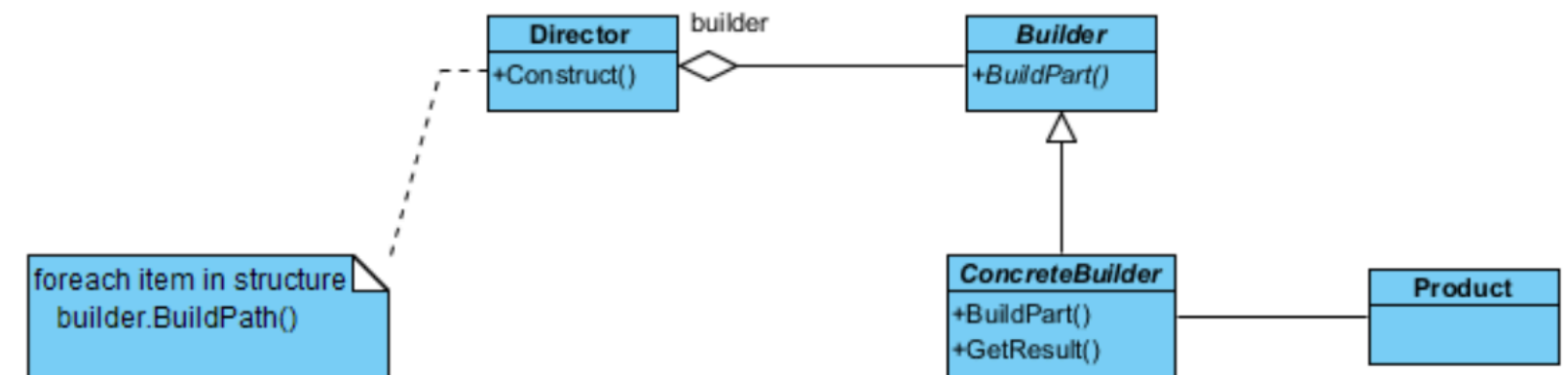
What is Builder?

Builder Pattern that constructs complex objects step by step, separating the construction process from the object's representation.

When is it commonly used?

- When an object has many attributes or initialization steps.
- When different representations of the same object are required.
- To avoid constructors with too many parameters.

Circle > UML Diagrams > GoF Design Patterns > Builder



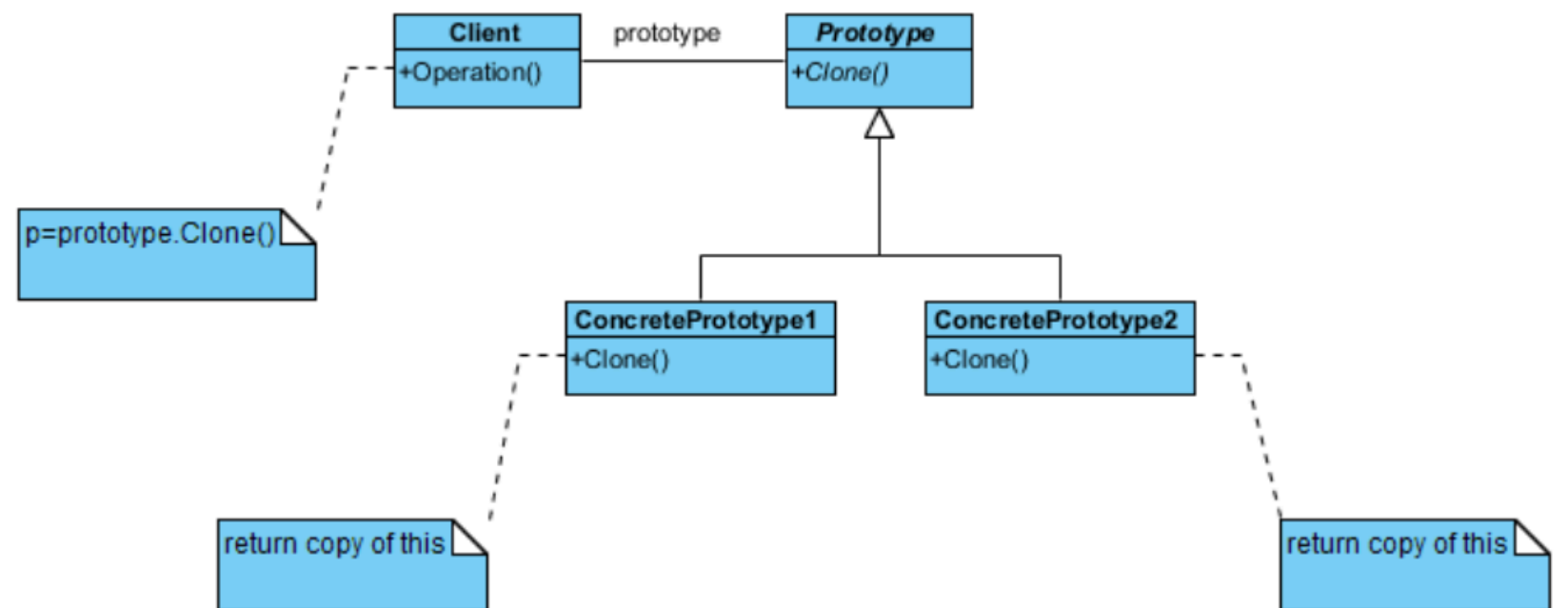
2. PROTOTYPE PATTERN

What is Prototype?

Prototype Pattern that creates new objects by cloning an existing object instead of creating them from scratch.

When is it commonly used?

- When object creation is expensive or time-consuming.
- When multiple objects share similar configurations.
- When fast object duplication is needed.



3. FACTORY METHOD PATTERN

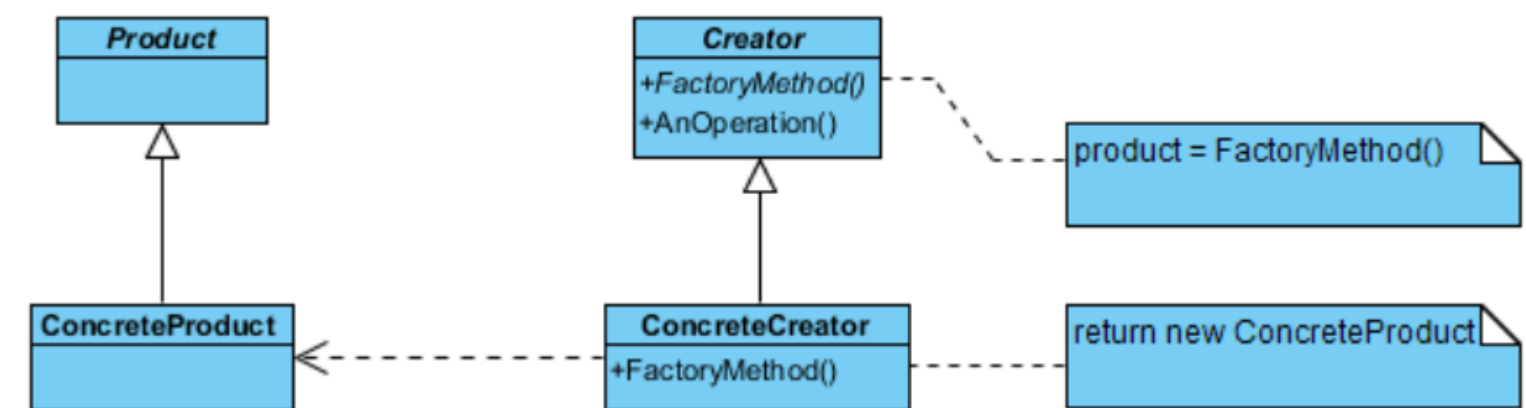
What is Factory Method?

Factory Method that provides an interface for creating objects, while allowing subclasses to decide which concrete object to instantiate.

When is it commonly used?

- When the client should not depend on concrete classes.
- When new object types need to be added easily.
- To support extensibility and follow the Open/Closed Principle.

e > UML Diagrams > GoF Design Patterns > Factory Method



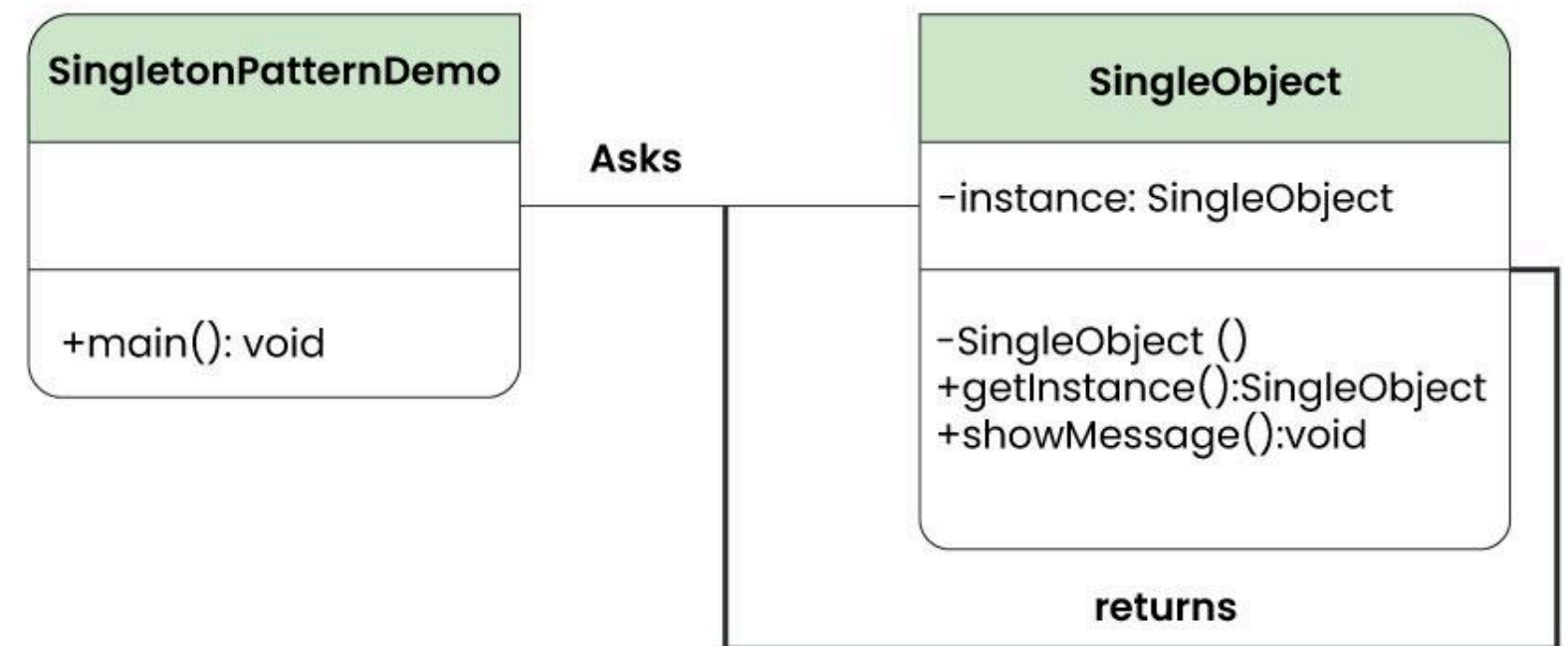
4. SINGLETON PATTERN

What is Singleton?

Singleton Pattern that ensures only one instance of a class exists and provides a global access point to that instance.

When is it commonly used?

- When only one shared instance is needed throughout the application.
- For managing shared resources.
- When a centralized object should be accessed globally.



Singleton Design Pattern in Java

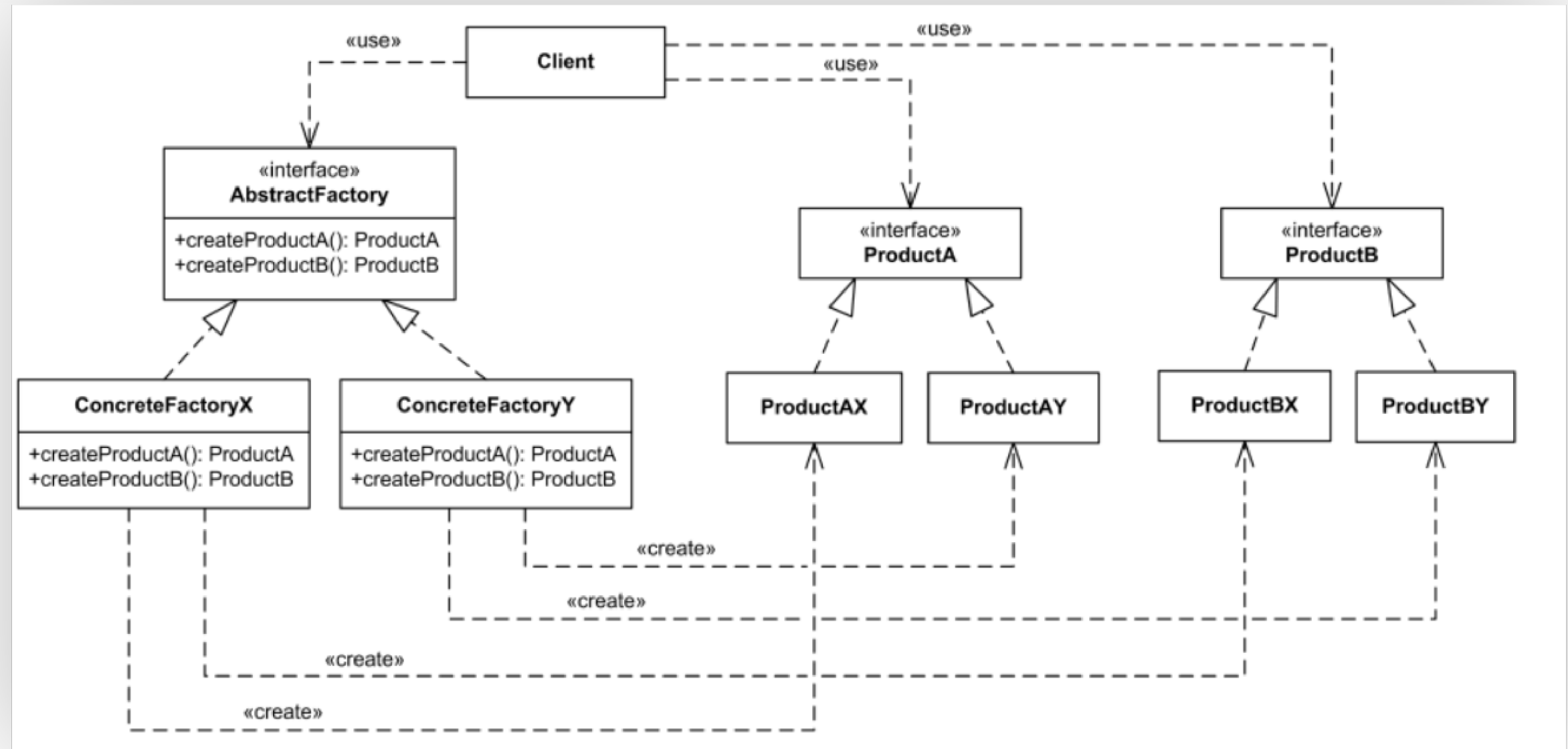
5. ABSTRACT FACTORY PATTERN

What is Abstract Factory?

- Category: Creational Design Pattern.
- Primary Goal: Allows you to produce families of related or dependent objects.
- Key Characteristic: Creates objects without specifying their concrete classes.
- How it Works: It provides an interface for creating these families, leaving the concrete implementation details to be handled by specific factory classes.

When is it commonly used?

- When a system needs to be independent of how its products are created, composed, and represented.
- When a system is configured with one of multiple families of products (e.g., UI elements for Windows vs. macOS).
- When you need to enforce that a family of related product objects is designed to be used together.

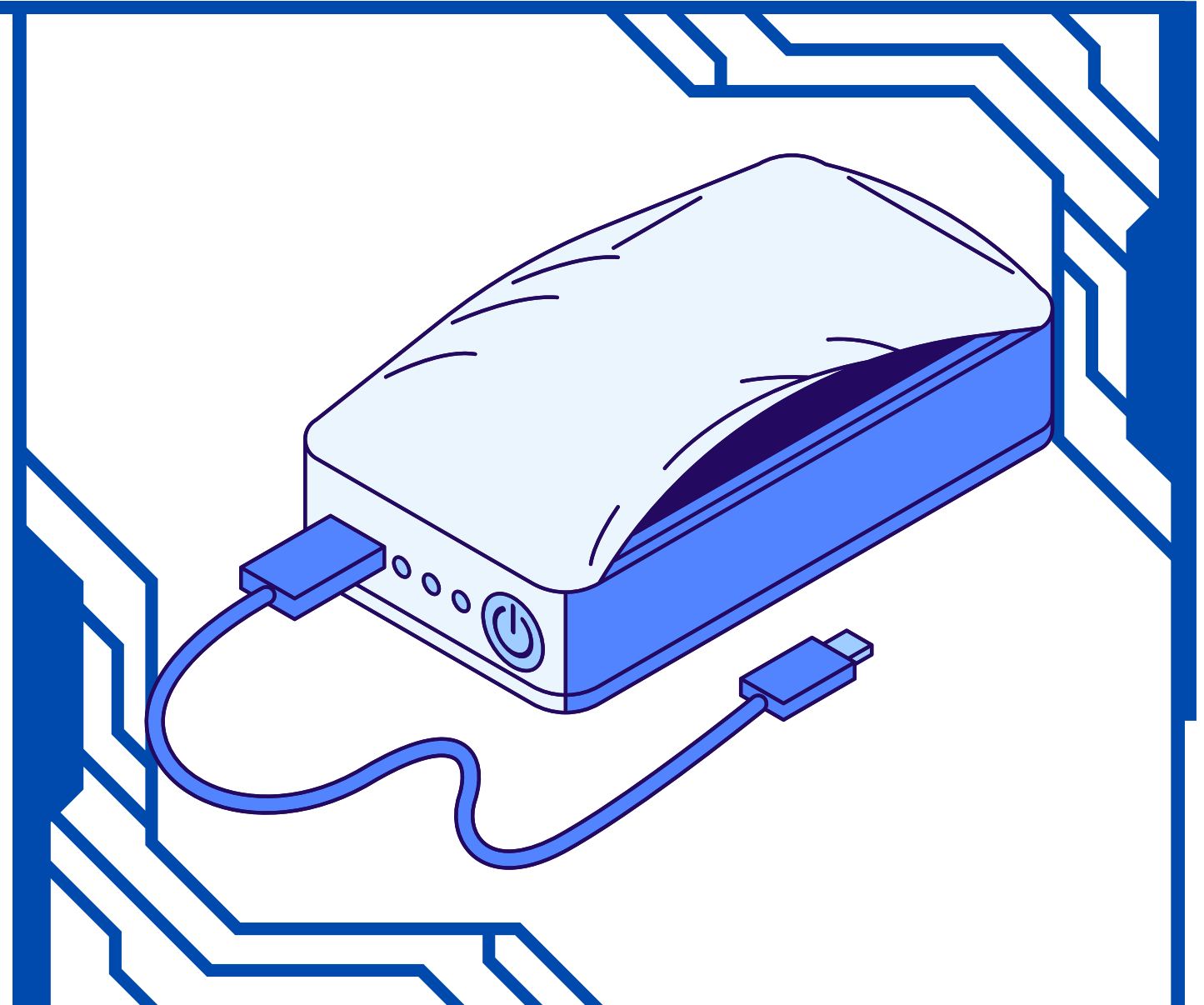


STRUCTURAL DESIGN PATTERNS

Deals with compositions of
objects and classes.

Main Purpose:

- How to compose objects into larger structures
- Keep the structures flexible
- Make it easy to modify relationships between objects



7 PATTERNS

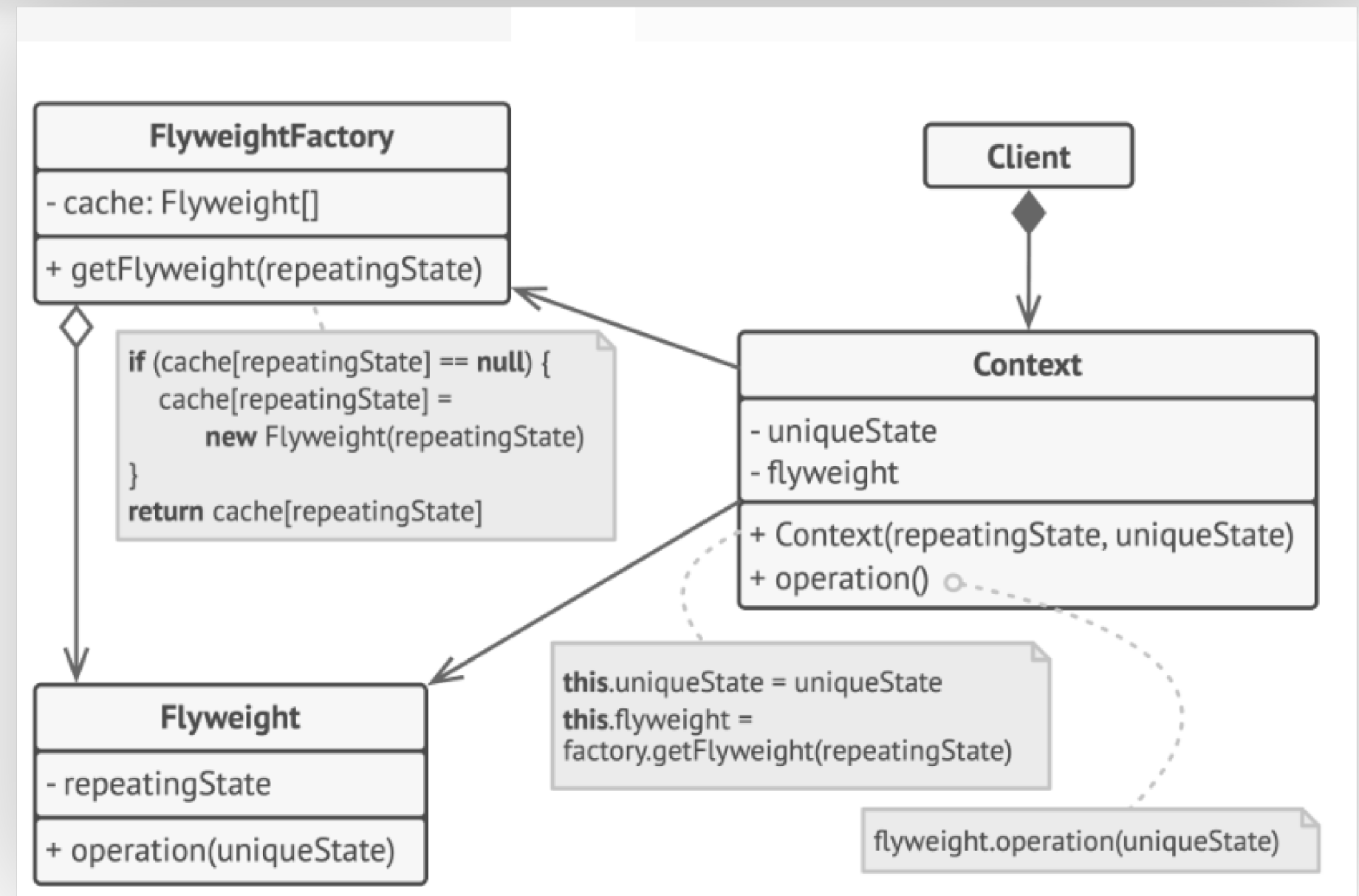
1. FLYWEIGHT PATTERN

What is FLYWEIGHT PATTERN?

- Category: Structural Design Pattern.
- Primary Goal: Minimizes memory usage by sharing as much data as possible with similar objects.
- How it Works: It separates an object's state into two distinct categories:
- Intrinsic State: Shared, invariant data that belongs to the object itself.
- Extrinsic State: Context-specific data that is passed in from the outside at runtime.

When is it commonly used?

- When an application uses an extremely large number of objects.
- When the storage costs are high because of the sheer quantity of objects.
- When most object state can be made extrinsic and managed outside the object itself (e.g., rendering thousands of trees in a game forest, where the tree mesh is shared, but coordinates are extrinsic).



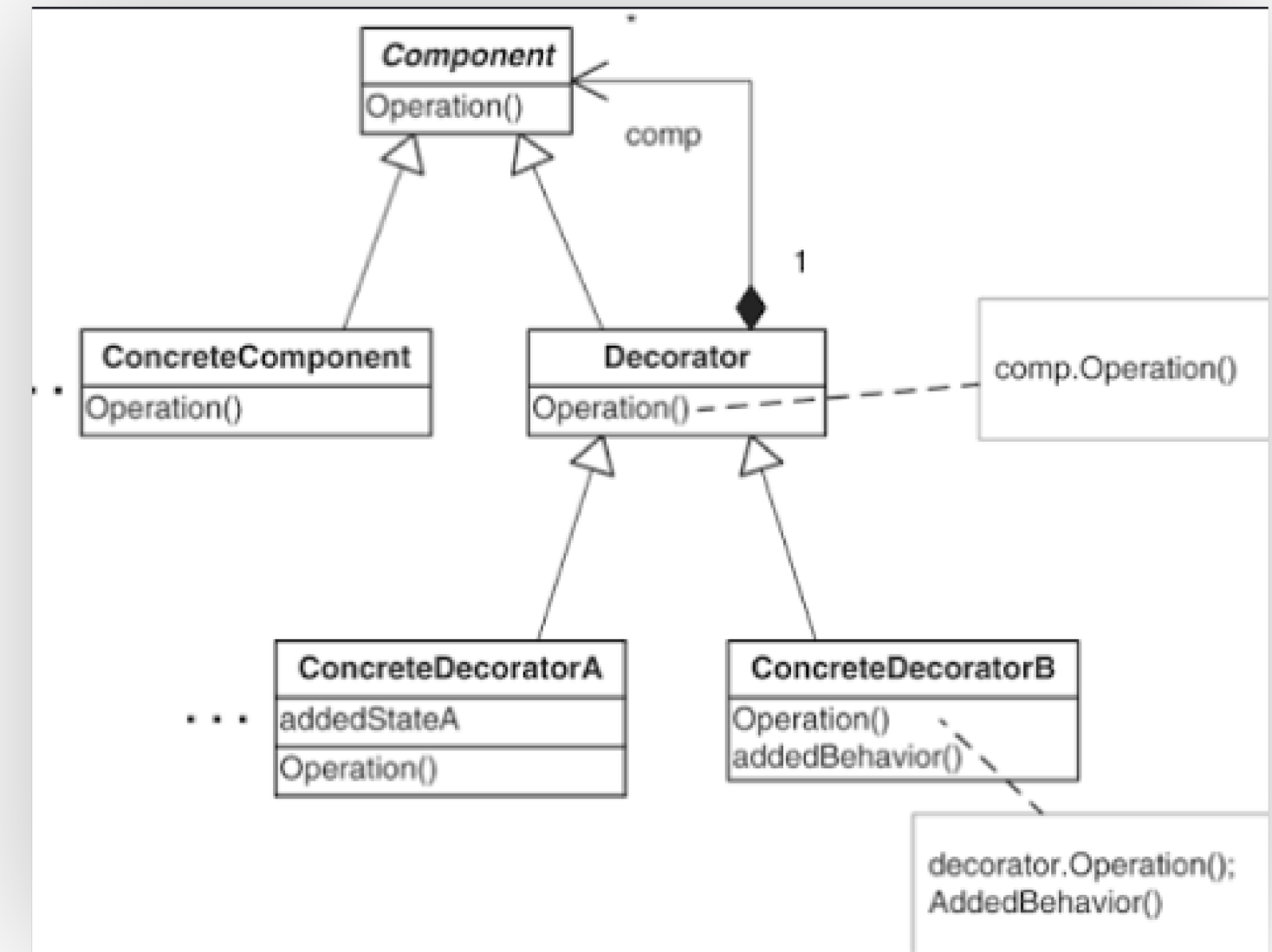
2. DECORATOR PATTERN

What is Decorator Pattern ?

- Category: Structural Design Pattern.
- Primary Goal: Allows you to dynamically attach new behaviors or responsibilities to an individual object.
- How it Works: Places the original object inside special wrapper objects that contain the new behaviors.
- Advantage: Provides a much more flexible alternative to subclassing when you need to extend functionality.

When is it commonly used?

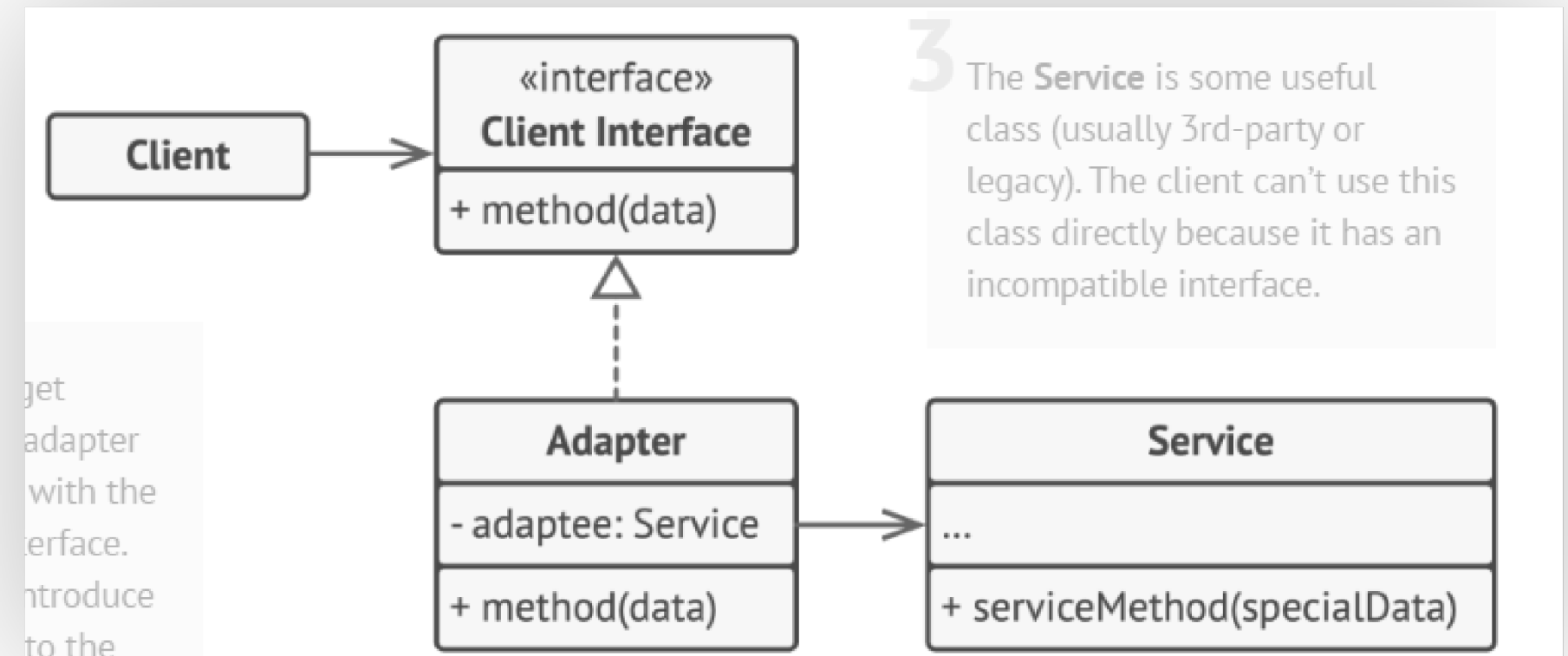
- When you need to add responsibilities to individual objects dynamically and transparently, without affecting other objects.
- When extending functionality through subclassing is impractical and would lead to an explosion of subclasses to support every combination of features.
- When you want to add features to objects (like adding scrolling, borders, or shadows to a UI component) on the fly



3. ADAPTER PATTERN

What is Adapter Pattern ?

- Category: Structural Design Pattern (also known as a Wrapper).
- Primary Goal: Allows objects with incompatible interfaces to collaborate and work together.
- How it Works: Acts as a bridge between two incompatible interfaces.
- Execution: It wraps one of the objects to completely hide the complexity of the conversion taking place behind the scenes.



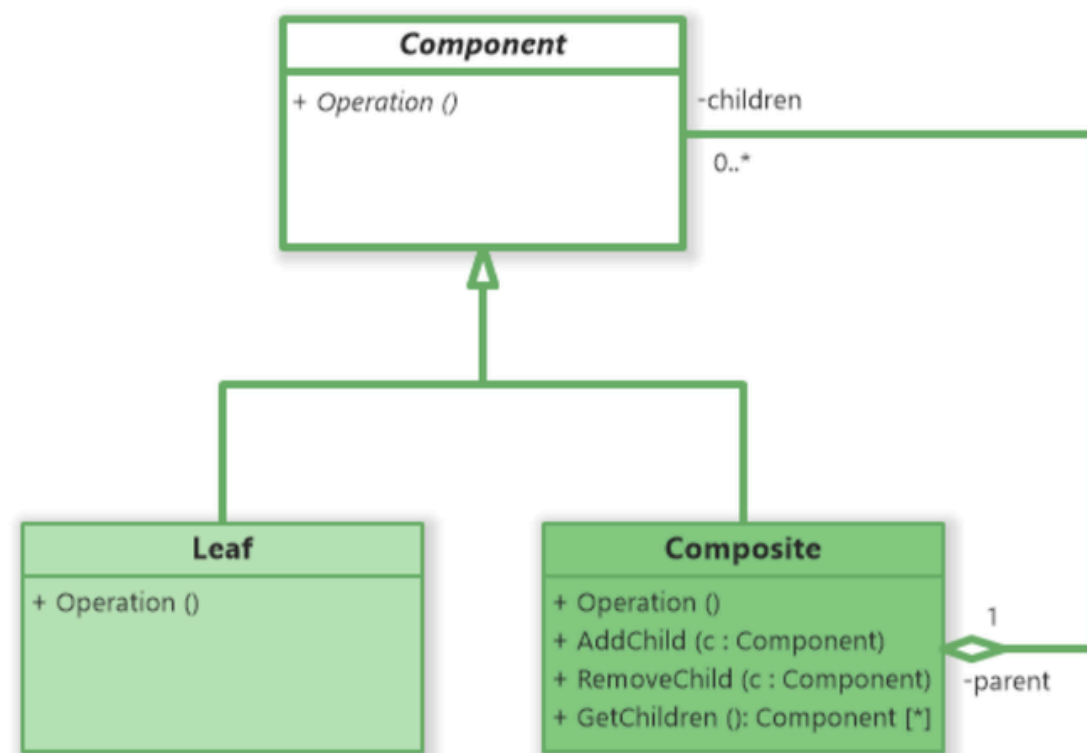
When is it commonly used?

- When you want to use an existing class, but its interface does not match the one you need (e.g., integrating a third-party library or legacy code).
-
- When you want to create a reusable class that cooperates with unrelated or unforeseen classes that don't necessarily have compatible interfaces.

4. COMPOSITE PATTERN

Composite is a structural pattern that lets you compose objects into tree structures and treat individual objects and compositions of objects uniformly

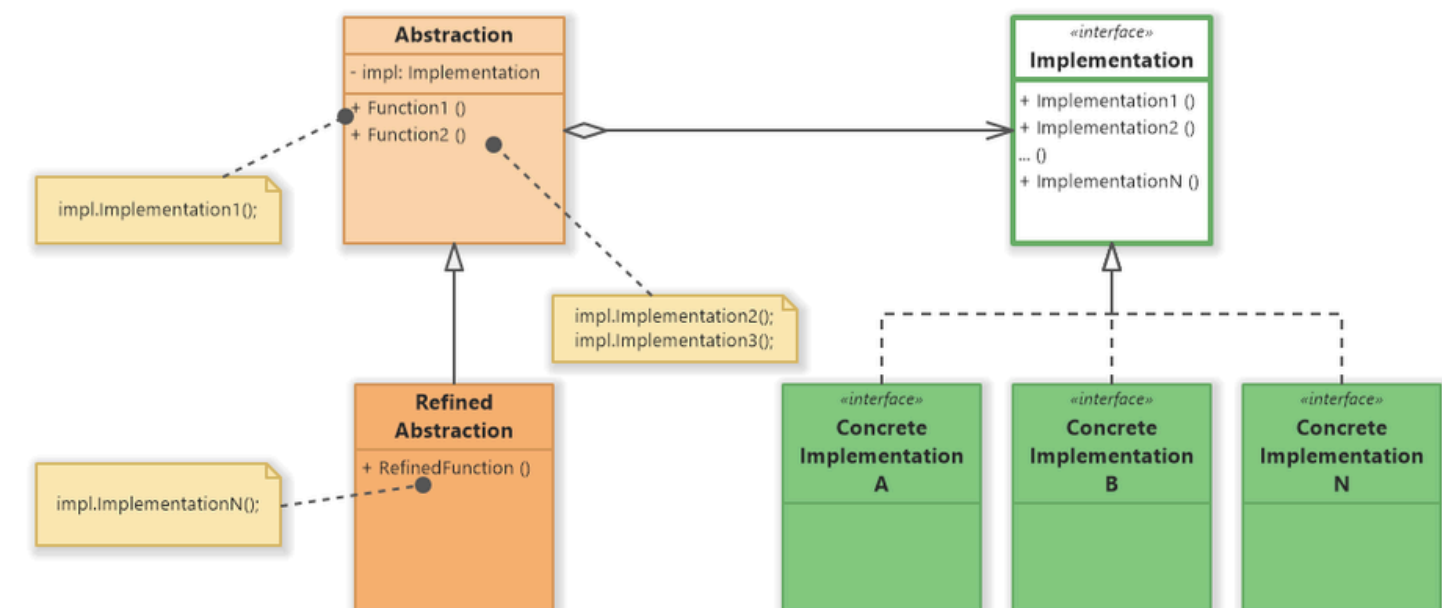
Used when: you need to represent part-whole hierarchies (e.g., file systems, UI components, org charts) and want clients to treat single objects and groups the same way



5. BRIDGE PATTERN

Bridge is a structural pattern that splits a large class (or closely related classes) into two separate hierarchies—abstraction and implementation—that can be developed independently.

Used when: you want to avoid a permanent binding between an abstraction and its implementation, or need to switch implementations at runtime (e.g., cross-platform UI, multiple device drivers).

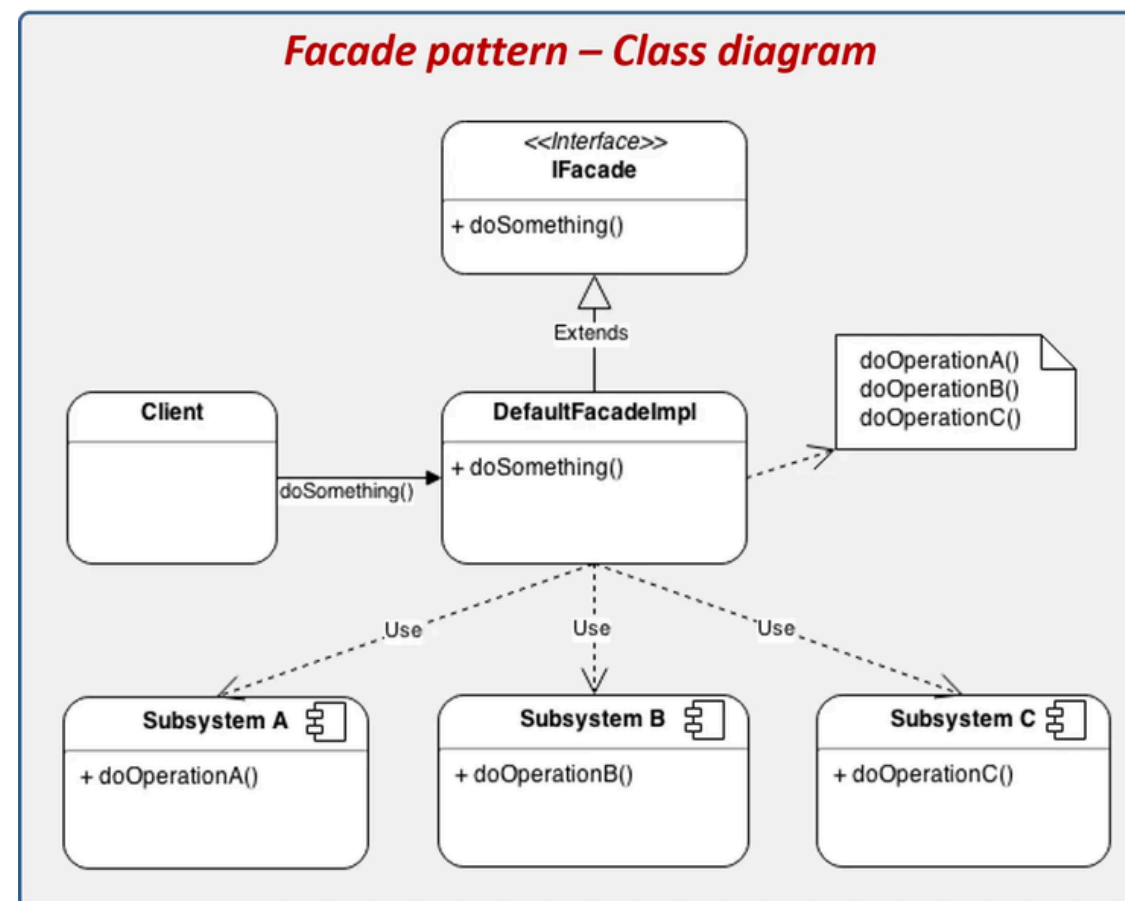


design pattern
Bridge

6. FACADE PATTERN

Facade provides a simplified, unified interface to a complex subsystem of classes.

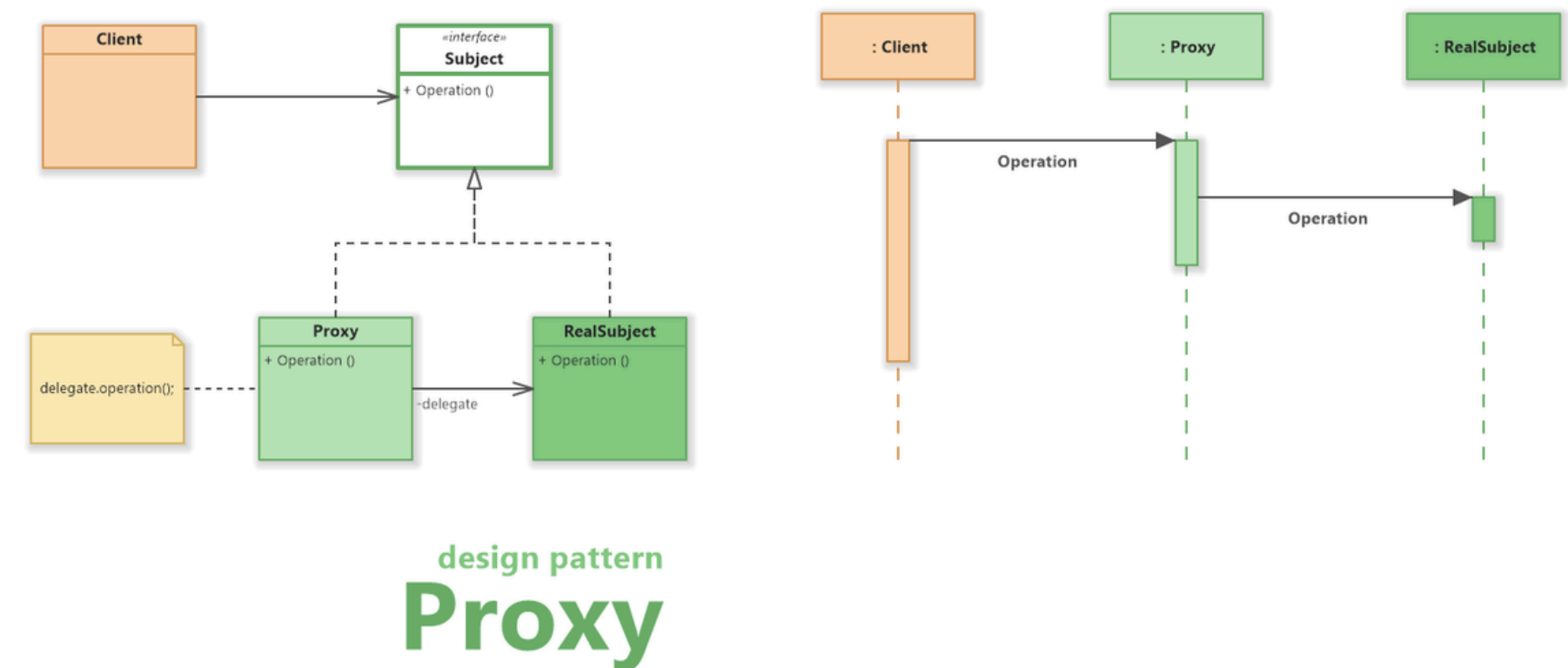
Used when: you want to reduce complexity for clients working with a large/complex library or set of subsystems (e.g., a single VideoConverter class hiding codec, audio, and file-handling subsystems).



7. PROXY PATTERN

Proxy provides a placeholder/surrogate object that controls access to another object.

Used when: you need lazy initialization, access control, logging, or caching without changing the original object (e.g., virtual proxies, protection proxies, remote proxies).

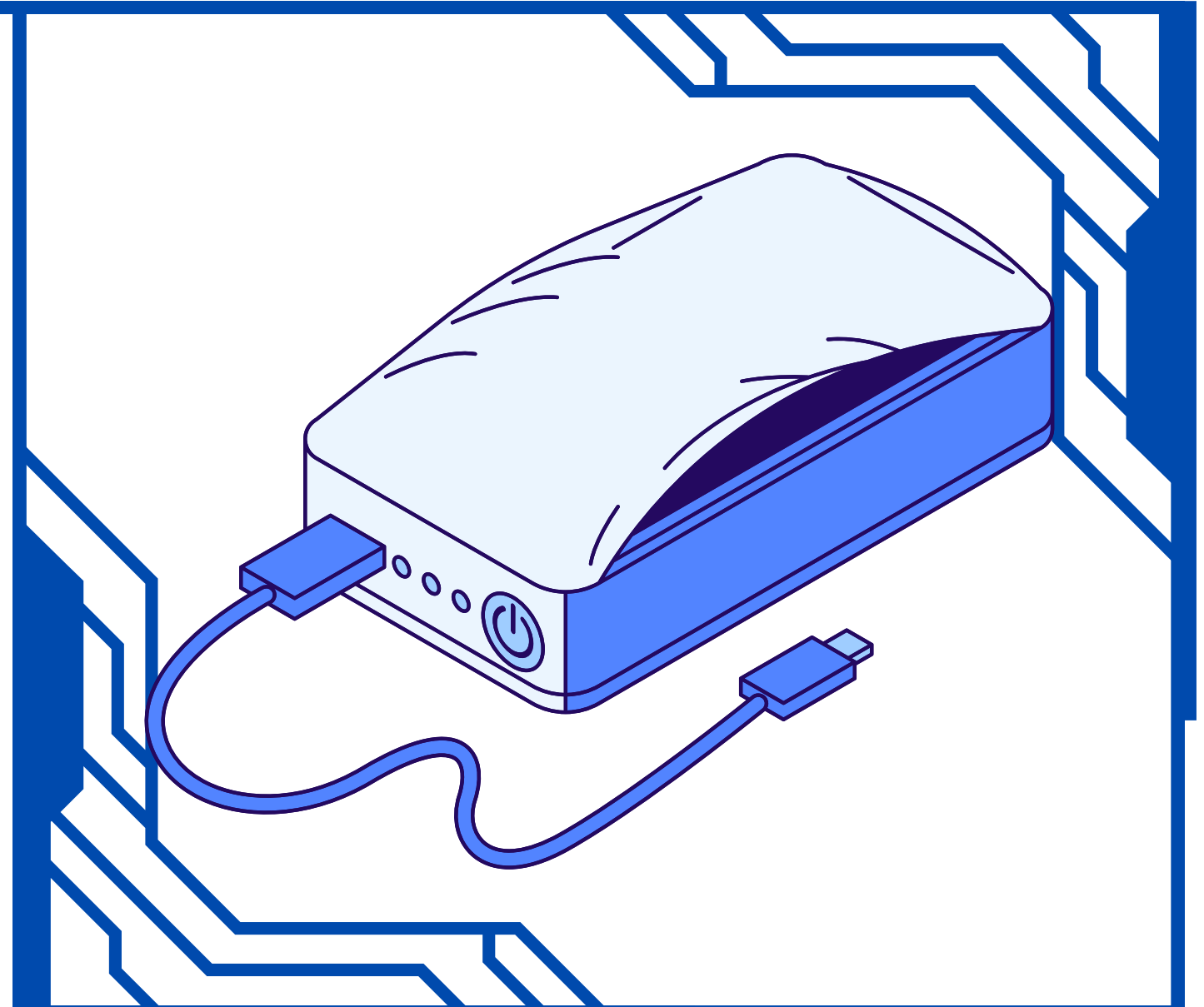


BEHAVIORAL DESIGN PATTERNS

Distribute responsibility between
classes and objects

Main Purpose:

- Communicate and distribute responsibilities
- Collaborate efficiently between objects
- Manage control flows and algorithms dynamically

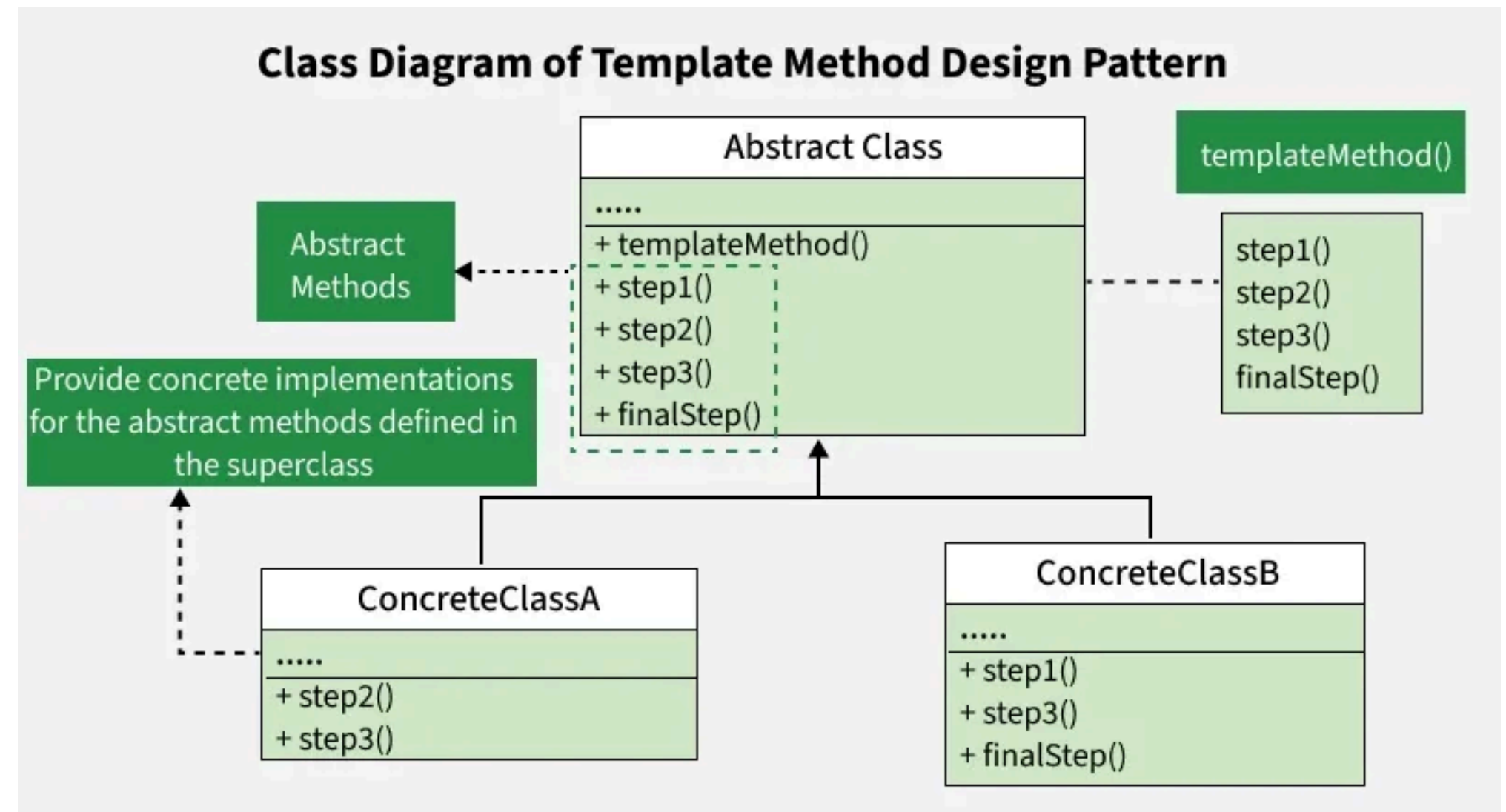


11 PATTERNS

1. TEMPLATE METHOD PATTERN

Template Method defines the skeleton of an algorithm in a base class, letting subclasses override specific steps without changing the algorithm's structure.

Used when: multiple classes share the same overall algorithm but differ in certain steps (e.g., data mining across CSV/PDF/DOC files, game AI turn sequences).



2.INTERPRETER

Defines a grammar for a simple language and a way to interpret sentences in it.

Use when: Need to evaluate a small, simple language/expression repeatedly (e.g., math expressions).

3.ITERATOR

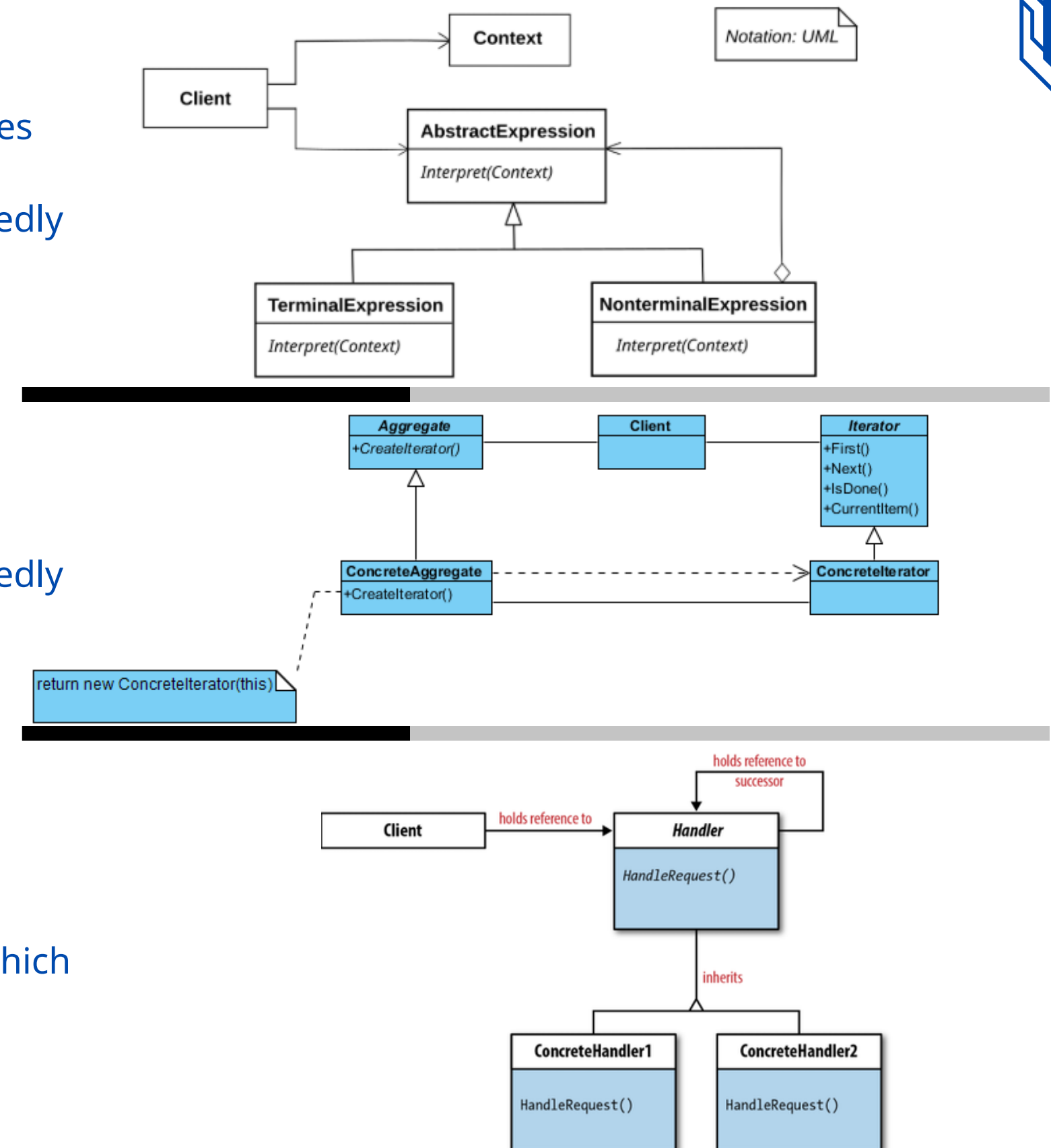
Lets you access elements of a collection sequentially without exposing its internal structure.

Use when: Need to evaluate a small, simple language/expression repeatedly (e.g., math expressions).

4.CHAIN OF RESPONSIBILITY

Passes a request along a chain of handlers, each deciding to process or forward it.

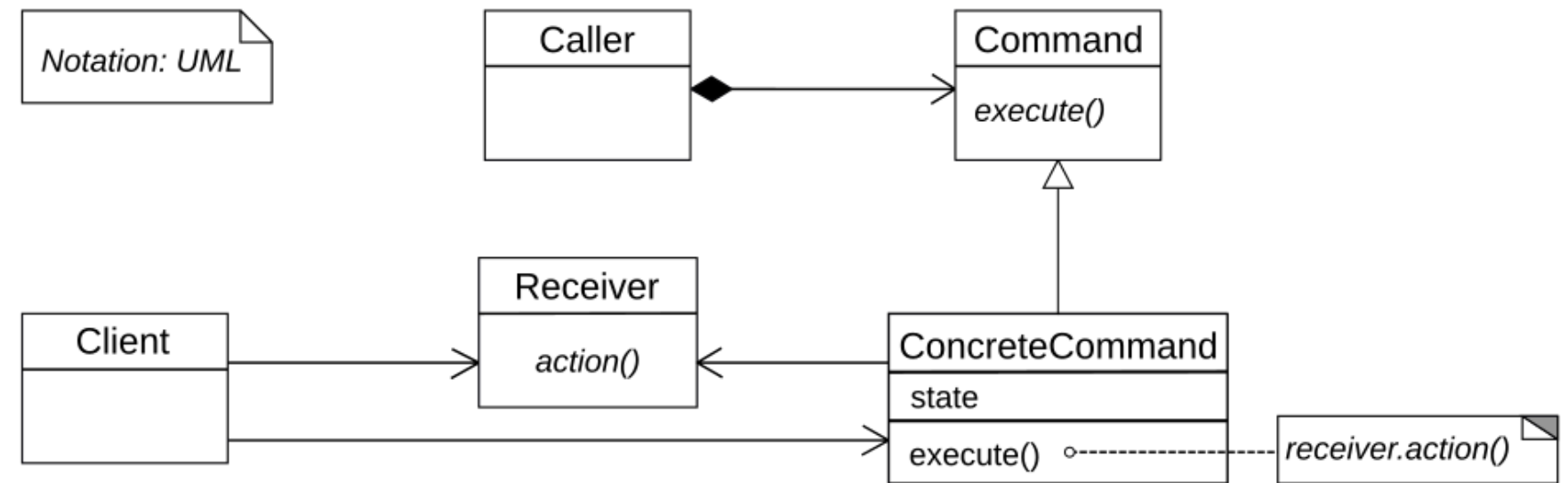
Use when: Multiple objects could handle a request but you don't know which one in advance; want to decouple sender/receiver.



5.COMMAND

Encapsulates a request (action + params) as a standalone object.

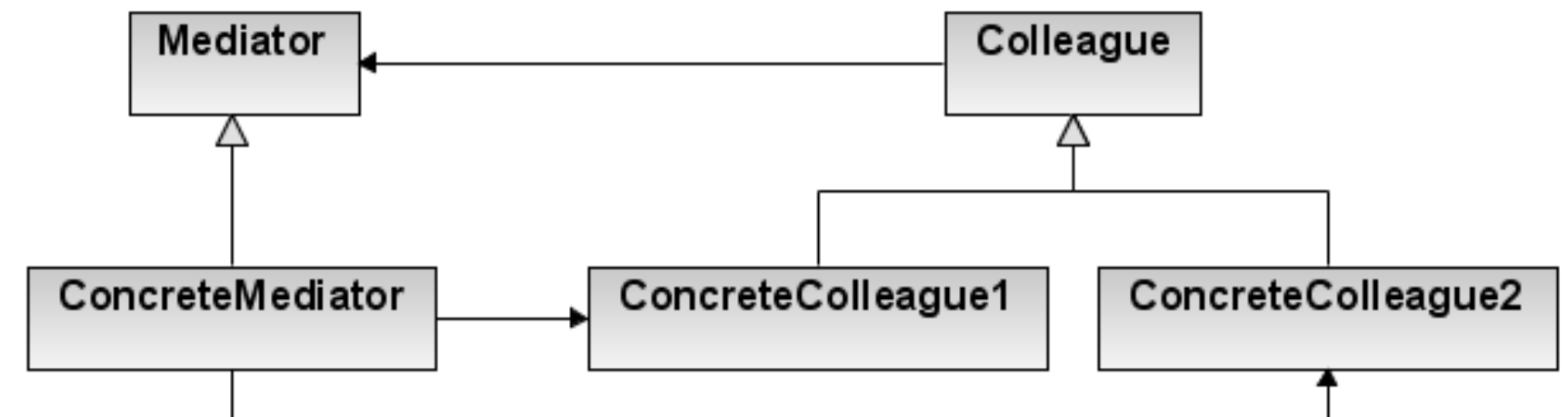
Use when: Need undo/redo, action logging, or queuing commands for later execution.



6.MEDIATOR

Reduces direct coupling between objects by routing their communication through a central mediator.

Use when: Many objects interact in complex ways, and direct links would create a tangled dependency web.

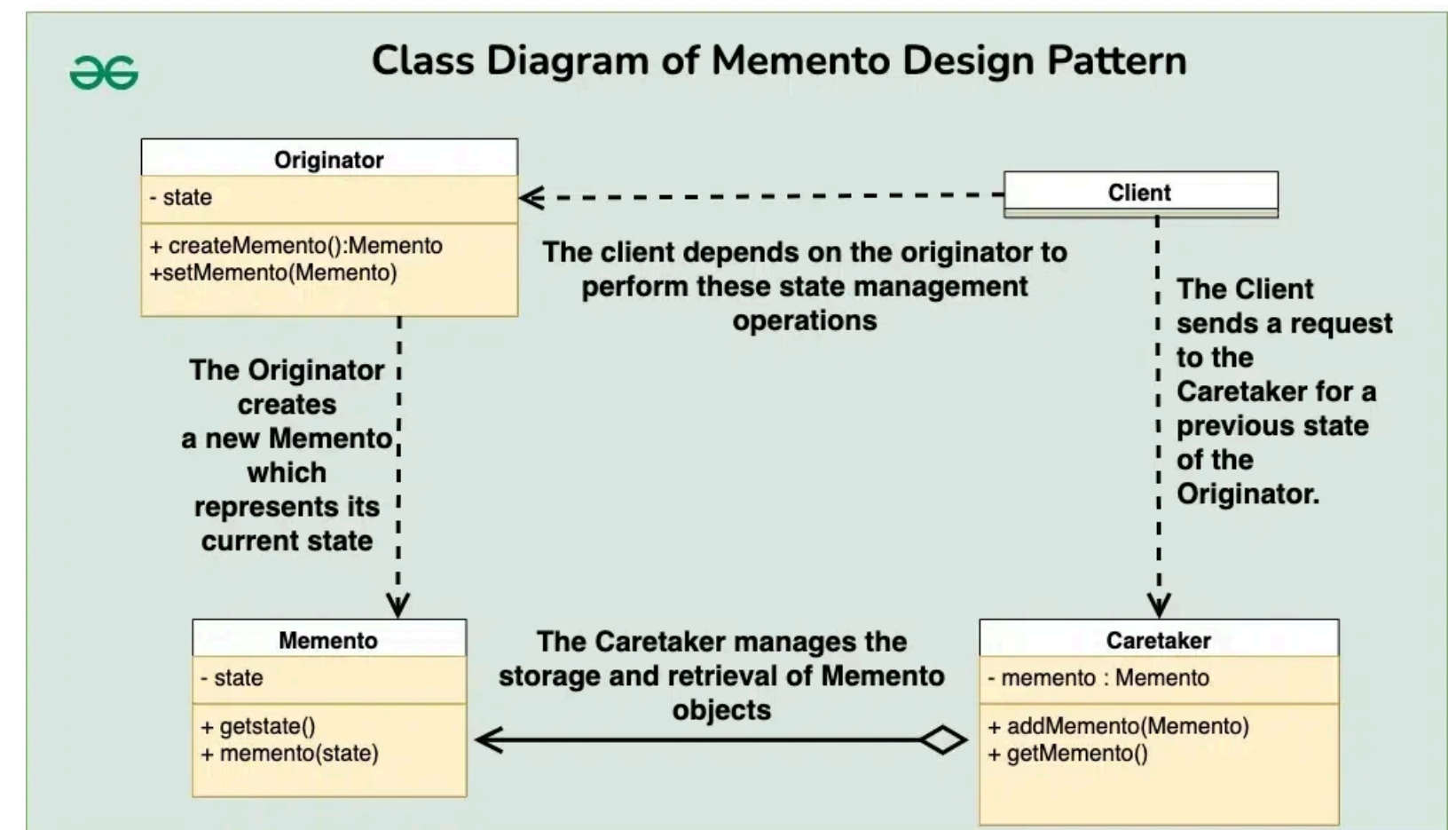


7. MEMENTO PATTERN

Saves and restores an object's state
(Undo/Redo)

Best for:

- Text editors
- Game saves
- Transaction rollbacks

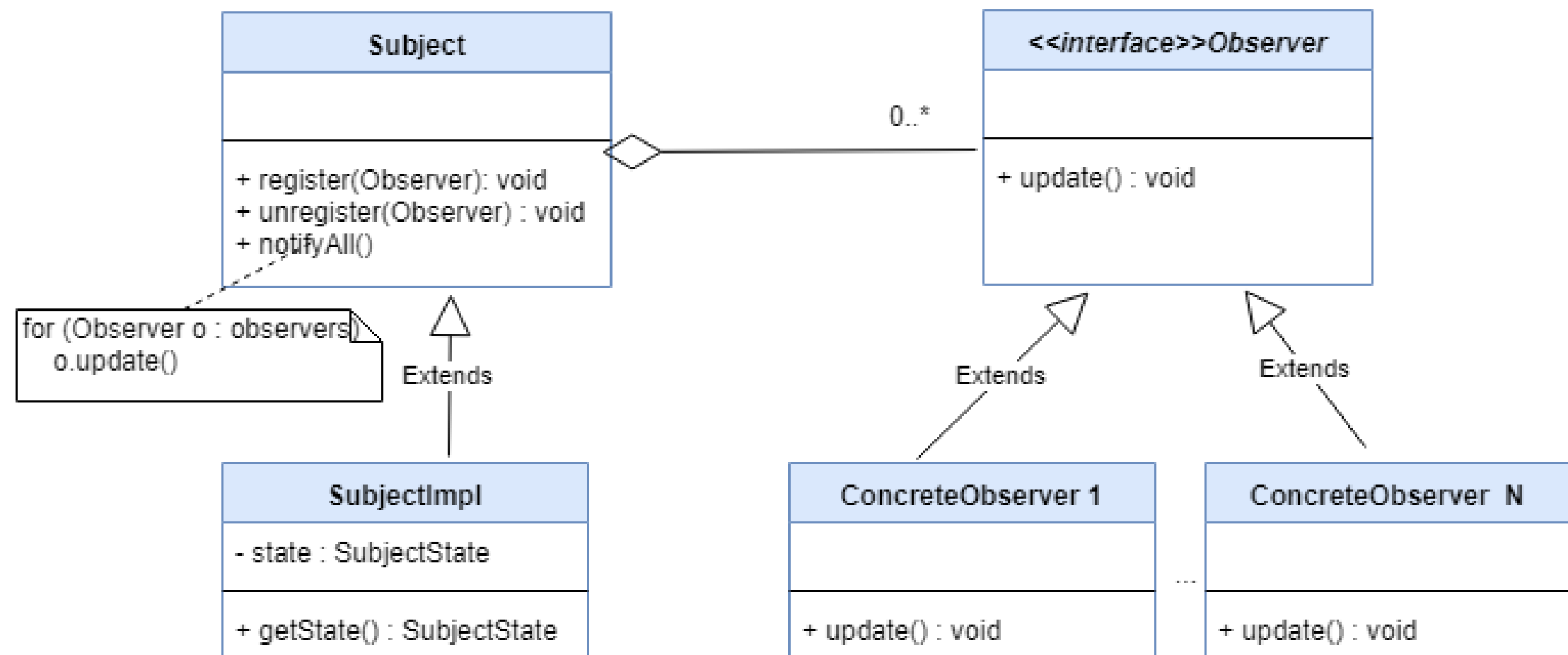


8. OBSERVER PATTERN

Notifies multiple dependent objects automatically when state changes

Best for:

- Event handling
- Notification systems
- UI updates

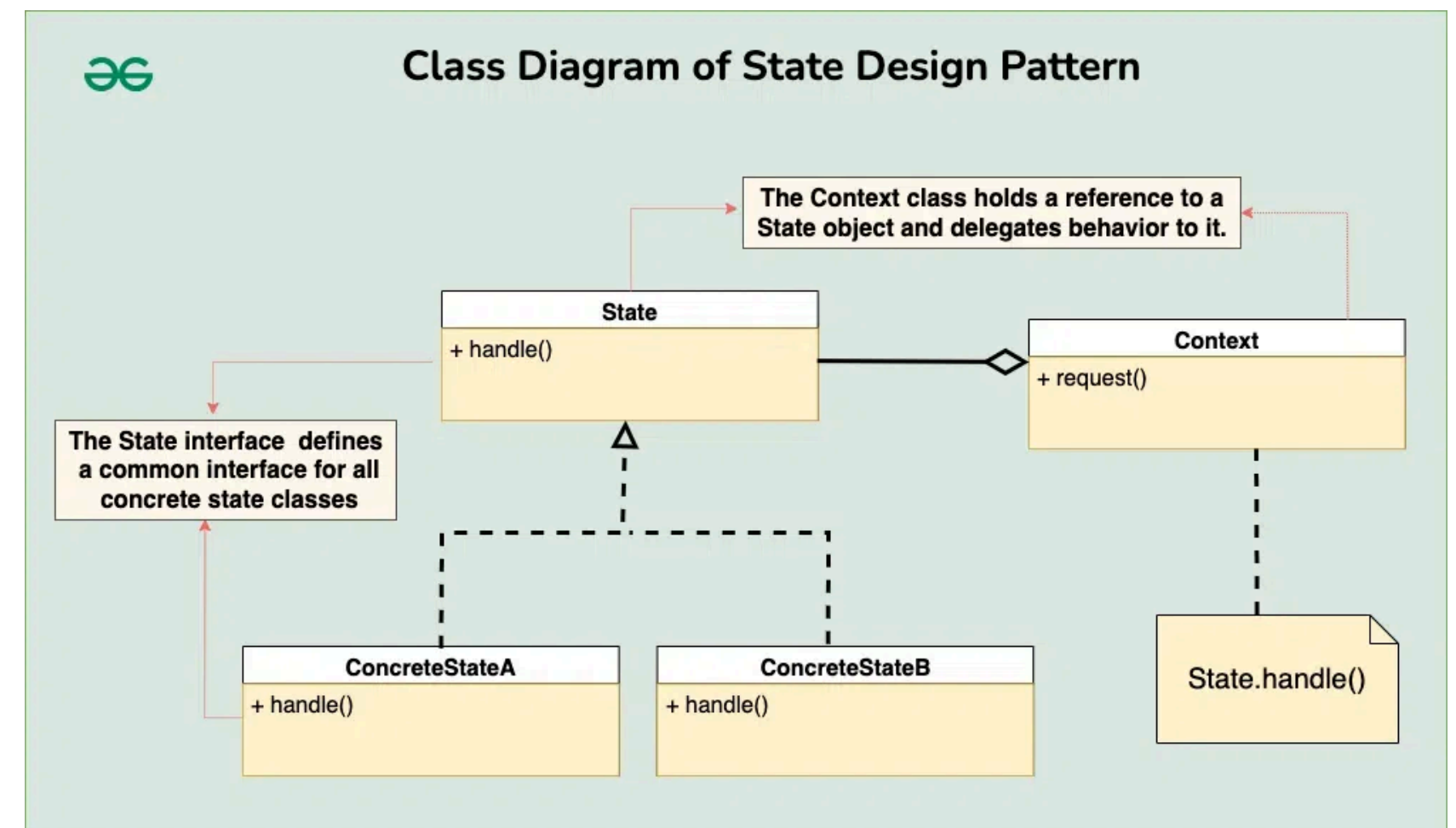


9. STATE PATTERN

Changes object behavior dynamically
based on its internal state

Best for:

- Workflow management
- Order tracking
- Vending machines

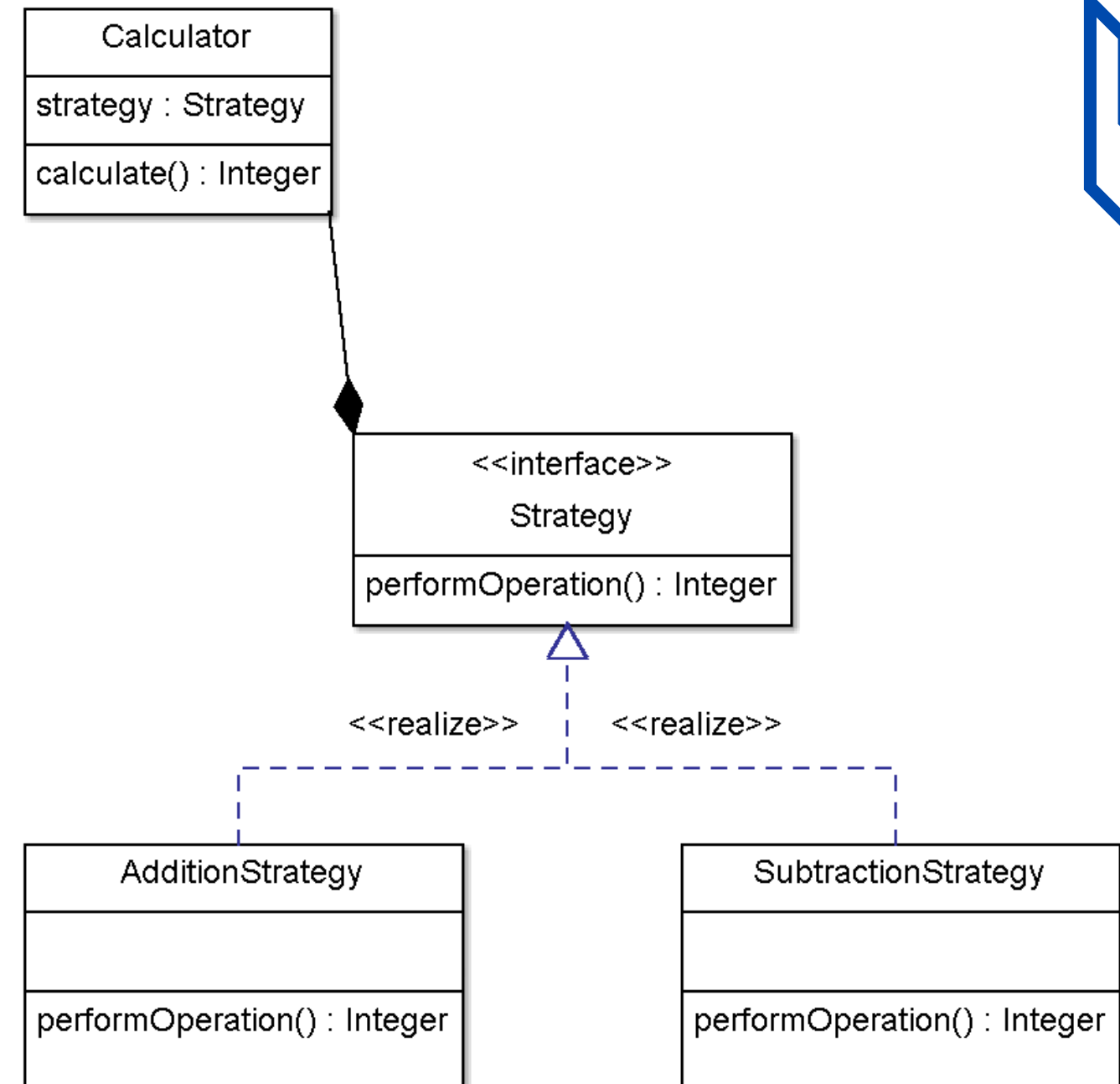


10. STRATEGY PATTERN

Swaps different algorithms
or rules flexibly at runtime

Best for:

- Payment gateways
- Routing options on maps

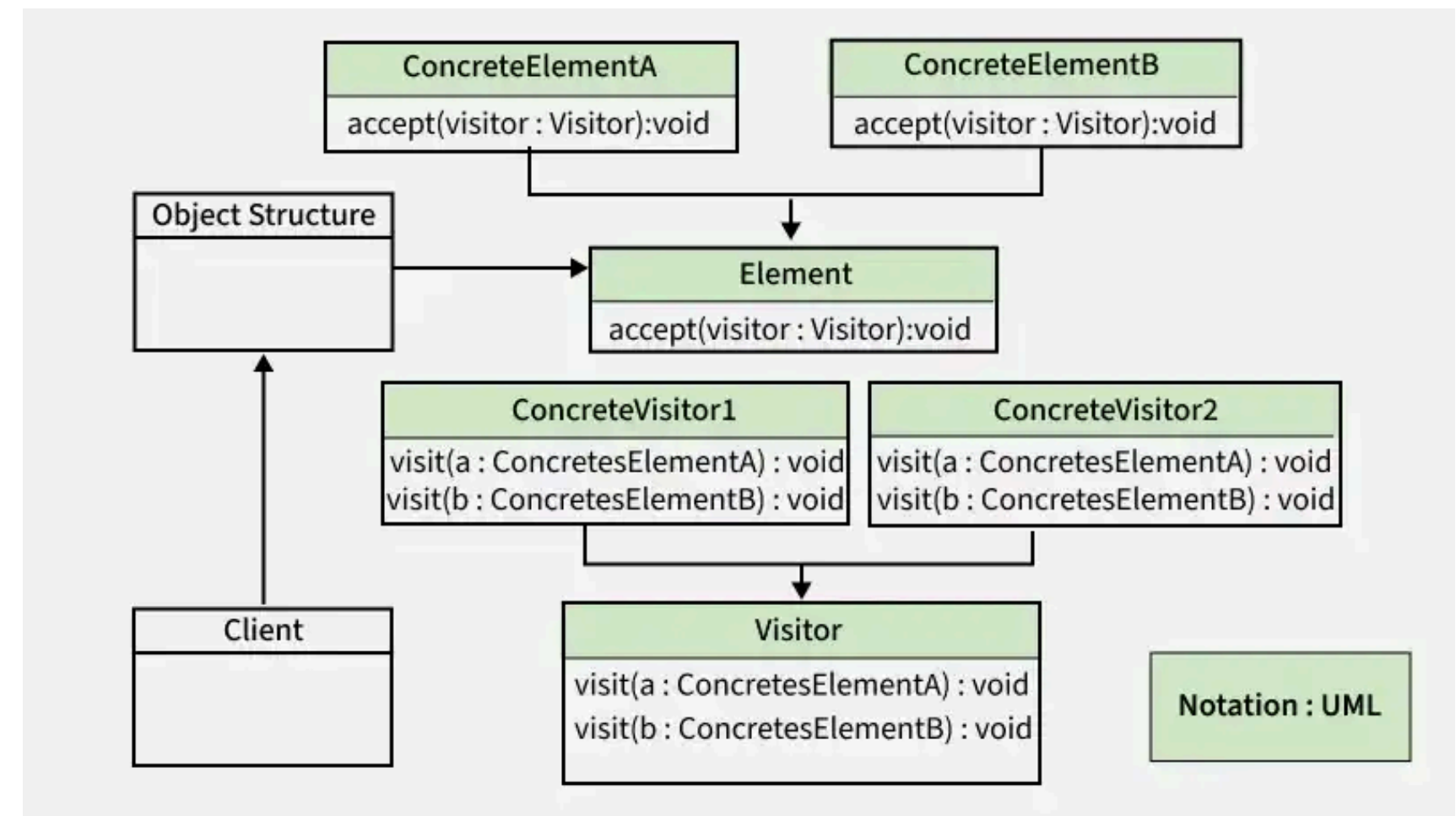


11. VISITOR PATTERN

Adds new operations to existing classes without modifying them

Best for:

- Document exporting (PDF/XML)
- Object structure reporting



DEMO

```
class Singleton {
  static instance = null;
  // constructor() {

  //   if (!Singleton.instance) {
  //     Singleton.instance = this
  //   }

  //   return Singleton.instance;
  // }

  static getInstance() {
    if (!Singleton.instance) {
      Singleton.instance = new Singleton();
    }

    return Singleton.instance;
  }

  someMethod() {
    console.log("Singleton method called");
  }
}

// const instance1 = new Singleton();
// const instance2 = new Singleton();
const instance1 = Singleton.getInstance();
const instance2 = Singleton.getInstance();

console.log("--- TEST 1: SINGLETON CƠ BẢN ---");
console.log(
  "Hai thực thể có bằng nhau không?",
  instance1 === instance2 ? "Có" : "Không",
);
instance1.someMethod();
```

client
instance 1, instance 2

getInstance()

Singleton

-instance: singleton

+getInstance(): singleton
+someMethod(): void

1



THANKYOU